

Writeup CTF : Flag Hunters

Plateforme : picoCTF | Catégorie : Reverse Engineering / Misc | Difficulté : Facile - Intermédiaire

Objectif / Description

Analyser le code source d'un mini-interpréteur codé en Python (un lecteur de paroles de chanson) afin de détourner son flux d'exécution et d'afficher le flag. Ce dernier est lu depuis un fichier local et chargé en mémoire au tout début de l'exécution, dans les premières lignes du texte manipulé par le programme.

Outils & Concepts clés

- Analyse statique et dynamique de code Python.
- Compréhension des mécanismes de pointeur d'instruction (Instruction Pointer / `lip`).
- Injection de séparateurs de commandes (`;`) et modification de flux de contrôle.
- Lecture et validation d'expressions régulières (Regex) en Python (`re.match`).
- Vulnérabilité d'évaluation de type *Second Order* (Données modifiées puis ré-évaluées).

Démarche pas à pas

1. Reconnaissance & Analyse Statique

En analysant le script Python, on remarque que le fichier `flag.txt` est lu dès le lancement et stocké dans la variable `secret_intro`, elle-même placée au tout début de la variable globale contenant la chanson complète (`song_lines[0]` à `song_lines[3]`).

Le programme agit comme une machine virtuelle (VM) basique ou un interpréteur : il boucle sur les lignes de la chanson grâce à un pointeur d'instruction nommé `lip` (Line Instruction Pointer). Les lignes de texte sont découpées par le caractère `;` via la fonction `.split(';')` avant d'être évaluées bloc par bloc par une série de conditions (`if/elif`).

2. Identification de la vulnérabilité

Il n'y a qu'un seul point d'interaction pour l'utilisateur, situé dans la condition `CROWD` :

```
elif re.match(r"CROWD.*", line):
    crowd = input('Crowd: ')
    song_lines[lip] = 'Crowd: ' + crowd
    lip += 1
```

Le code présente une faille critique de persistance en mémoire (absence de sanitization) : notre saisie utilisateur écrase directement la ligne courante dans le tableau global `song_lines`. Comme la chanson effectue des boucles récurrentes grâce au mot-clé `REFRAIN`, la ligne que nous venons de modifier sera de nouveau lue et **réinterprétée** par le script lors du deuxième passage.

3. Exploitation (Création du Payload)

Pour afficher le flag (situé à l'index 0), nous devons exploiter l'instruction interne de saut conditionnel `RETURN [nombre]`, qui permet d'écraser la valeur de la variable `lip`. La syntaxe exacte attendue par l'interpréteur est imposée par la regex : `re.match(r"RETURN [0-9]+", line)`. Notre objectif est de forcer l'exécution de l'instruction `RETURN 0`.

Pourquoi l'injection directe `;RETURN 0` échoue-t-elle ?

Si l'on injecte uniquement `;RETURN 0`, la ligne en mémoire devient `Crowd: ;RETURN 0`. Lors de l'évaluation au passage suivant, le `split(';')` isole le premier élément : `"Crowd: "`.

Cette chaîne valide silencieusement la condition `CROWD.*`, ce qui déclenche immédiatement l'instruction `lip += 1`. Le pointeur passe donc à la ligne suivante en mémoire

avant

que le script n'ait eu l'occasion d'évaluer correctement le second bloc `"RETURN 0"`.

Pour contourner la logique du script et stabiliser l'exploit, l'injection doit être construite en trois parties :

- La première partie (ex: `some_string`) permet de "consommer" la condition `CROWD.*` proprement, ce qui déclenchera un `lip += 1` qui sera écrasé plus tard.
- Le séparateur `;` permet de clôturer la première instruction de façon native.
- La seconde partie de l'injection (`RETURN 0`) sera lue au cycle de boucle suivant, et écrasera la variable `lip` en la forçant à `0` de façon définitive.

4. Obtention du flag

Lors de la première invite de commande `Crowd:`, nous injectons la payload suivante :

```
some_string;RETURN 0
```

Au passage suivant dans le refrain, la ligne est découpée. L'interpréteur traite d'abord `Crowd:` `some_string`, incrémente `lip`, puis traite immédiatement l'instruction `RETURN 0`, ce qui force `lip = 0`. Le programme ré-affiche les toutes premières lignes de la chanson et révèle le flag : `picoCTF{70637h3r_f0r3v3r_a5202532}`.

Leçons retenues & Automatismes à garder

- **Identifier le Pointeur d'Instruction (IP/PC)** : Dans tout challenge de Reverse Engineering impliquant un environnement sandboxé, un interpréteur ou une VM personnalisée, la priorité absolue est d'identifier la variable qui contrôle le flux d'exécution (ici `lip`) et de chercher par quels moyens il est possible de l'altérer.
- **Traquer les vulnérabilités du "Second Order"** : Si une entrée utilisateur est stockée en mémoire de manière non sécurisée puis réutilisée/réévaluée par l'application ultérieurement, cela constitue une surface d'attaque idéale pour des injections différées complexes.
- **L'importance du fuzzing manuel des séparateurs** : Toujours tester les caractères de contrôle (`;`, `|`, `&&`) lors des audits d'inputs pour vérifier la robustesse des opérations de parsing (découpage vulnérable).